

Binary Image Classification Task with Basic Classifiers and CNN based models on ISIC 2016 dataset and SAR-CoV-2 Dataset



Course : INSE 6180
Winter 2025

DEEP BHATTACHARYYA
40299945

Contents of The Presentation

- 1) Introduction
- 2) Problem Statement
- 3) Basic Classifiers used
- 4) Naïve Bayes Algorithm
- 5) KNN Algorithm
- 6) Convolution Neural Network Models used
- 7) AlexNet Architecture
- 8) MobileNet-V2 Architecture
- 9) DenseNet201 Architecture
- 10) ResNet101 Architecture
- 11) ViT-B-16 Architecture
- 12) Experimental Results
- 13) Best Outcomes
- 14) Conclusion
- 15) Further Opportunities and Ideas



INTRODUCTION

Medical Image classification is a very important task in the field of medical imaging and computer vision. It involves automatically assigning labels to medical images, such as identifying whether a lesion is benign or malignant, or classifying chest X-rays as normal or abnormal.

Classifying types of skin lesions or covid types with good model performance can have significant advantage to doctors in making quick decisions. This will ultimately lead to being an efficient approach in taking the further required actions in curing the disease and speed up the process.



Problem Statement

This project aims to apply basic classifiers and Convolution Neural network-based models on ISIC 2016 dataset and SARs-CoV-2 dataset as to why CNN performs better in medical images.



Basic Classifiers Used

The basic Classifiers used in this project are:

- 1) Naïve Bayes Classifier
- 2) K-Nearest Neighbour Classifier



Naïve Bayes Algorithm

The Algorithmic approach for Naïve Bayes:

- 1) First we preprocess the Images by converting them into size of 224x224x3.
- 2) Then we use the raw pixels for the feature extraction(many other methods like colored histograms, LBP of image) that is to flatten them into 1D feature vectors.
- 3) Then the preparation of the Dataset done that is to split it into training and testing.
- 4) Then we build the Naïve Bayes Classifier.
- 5) Lets say an the Input to the classifier is Feature matrix X of shape (n, d) and label vector y of length n .
- 6) Now we calculate the Probabilities for the class labels(Prior probabilities) using the Gaussian distribution since the values are Continuous.
- 7) Then we calculate the Probabilities $P(X_i|C_i)$ where feature matrix is X and Class label is C .
- 8) Then we calculate the Posterior probability for the test data .
- 9) Now we store the predicted labels for a test data .
- 10) Evaluate the predicted labels based on the performance metrics.



K-Nearest Neighbour Algorithm

The Algorithmic approach for K-Nearest Neighbor:

- 1) First we preprocess the Images by converting them into size 224x224x3.
- 2) Then we use the raw pixels for the feature extraction(many other methods like colored histograms, LBP of image) that is to flatten them into 1D feature vectors.
- 3) Then the preparation of the Dataset is done that is to split it into training and testing
- 4) Then we build the K-Nearest Neighbor classifier.
- 5) Then we define the number of K nearest neighbors.
- 6) Storing the data in training memory.
- 7) Now to predict the new images, we compute the Euclidean distances between testing image and all training images.
- 8) Then we select the k nearest training samples.
- 9) Perform majority voting among the k neighbors to assign a class label.
- 10) Predict labels for test images
- 11) Evaluate the predicted labels based on the performance metrics.



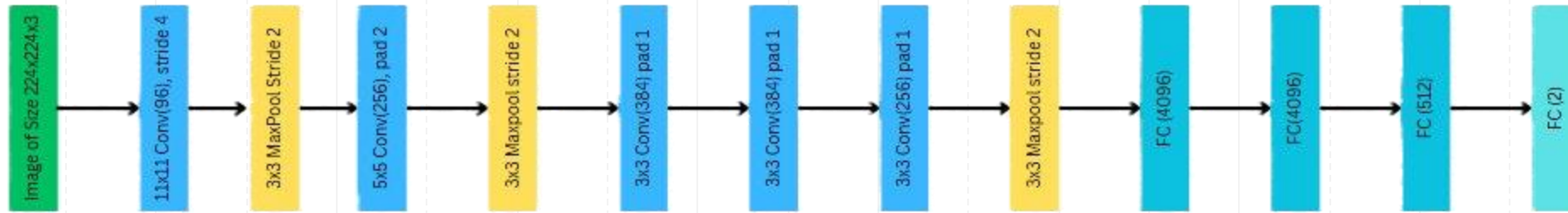
Convolution Neural Network Models Used

The CNN-models used in this project are:

- 1) AlexNet
- 2) DenseNet-201
- 3) MobileNet-V2
- 4) ResNet-101
- 5) ViT-B-16

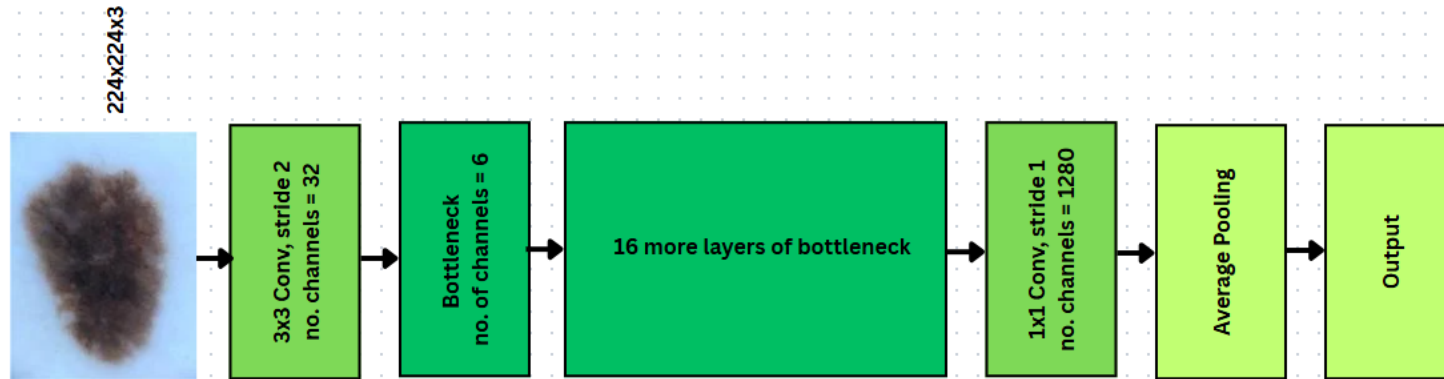


AlexNet Architecture

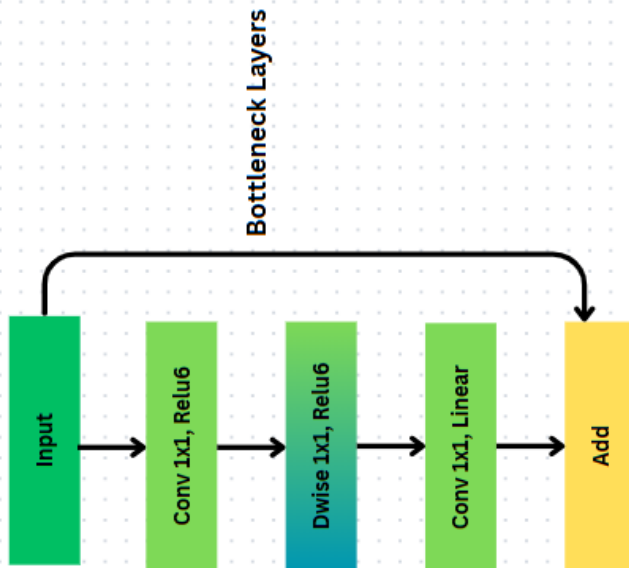


- 1) First we take an input image of size 224x224x3
- 2) Then we perform 11x11 Convolution with a stride length 4 to detect patterns like edges, textures and shapes.
- 3) These are then followed by max pooling layer of stride 2 to reduce the length of the data while keeping the important features.
- 4) Then it is followed by 6 more layers which consists of 4 convolution layers and 2 max pooling layers.
- 5) As the input is flattened it is passed through the fully connected layers which makes the decision after all the important features from the image has been extracted and delivers the output based on the extracted information to which class label it belongs.

MobileNet-V2 Architecture

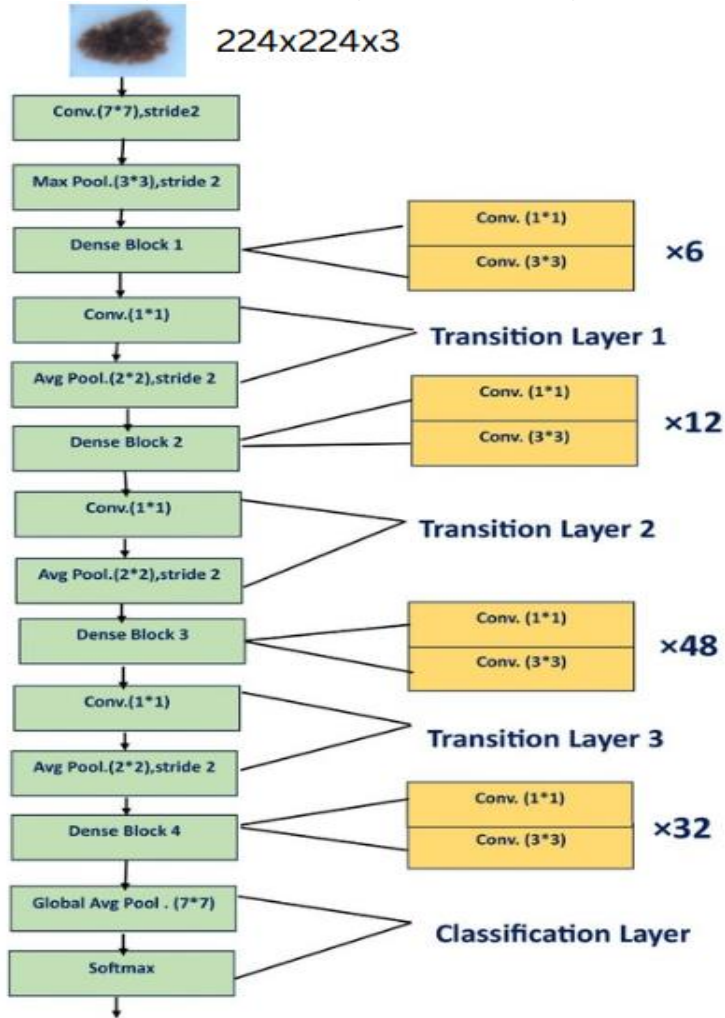


- 1) First we take an input image of size 224x224x3.
- 2) Then we perform 3x3 convolution of stride 2 with number of channels being 32 to extract localized features like edges, colors.
- 3) Then the image goes through a series of bottleneck layers.



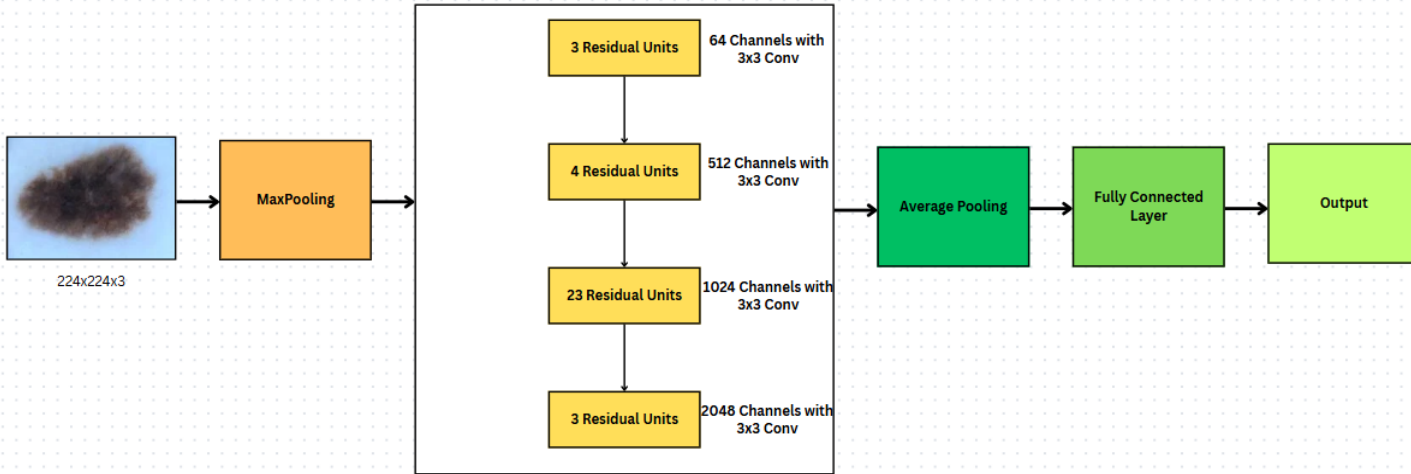
- 4) The bottleneck consists of 3 more layers first it expands in layer and then applies depthwise convolution to filter each channel separately and projecting it back to a smaller size.
- 5) These steps help reduce computation while preserving important information. Around 17 such bottleneck layers are stacked to learn complex patterns.
- 6) After that, a 1x1 convolution with 1280 channels is used to combine features, followed by average pooling to reduce dimensions. Finally, the result is passed to a fully connected layer that gives the final prediction.

DenseNet-201 Architecture

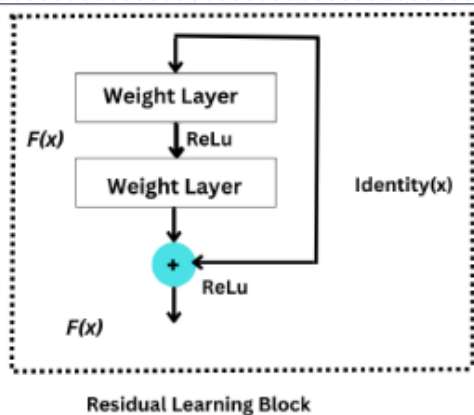


- 1) First we take an input image of size 224x224x3 which is processed through a 7x7 convolution layer with stride 2 to extract basic features like edges, local textures, followed by max pooling that reduces the dimension but protects the crucial features.
- 2) Then the image enters dense block1 which consists of 6 convolution blocks with different size 1x1 and 3x3 all the layers are connected so that each layer receives the feature from the previous layer.
- 3) After that, there is a transition layer with 1x1 convolution and average pooling, which reduces the spatial dimensions and passes the output to dense block 2.
- 4) Now again this dense block contains 12 layers which are connected and then it is followed by the transition layer 2.
- 5) This process continues and we have 2 more Dense blocks and another transition layer.
- 6) After all this process we come to the classification layer where the final prediction is made.

ResNet-101 Architecture

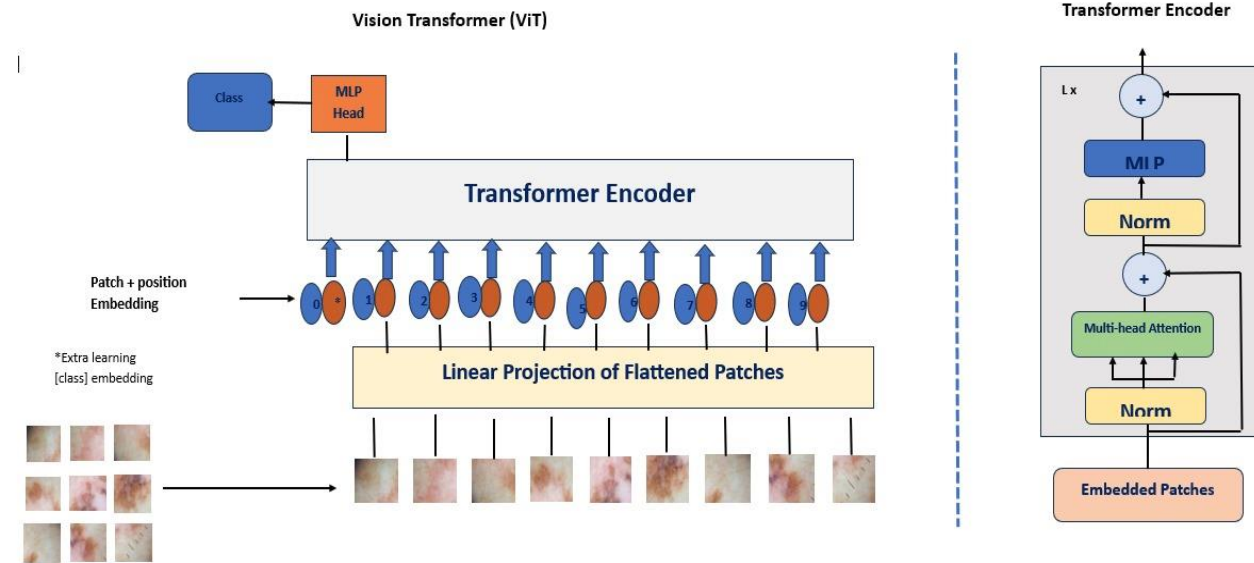


- 1) First we take an input image of size 224x224x3.
- 2) Then we apply a max pooling operation to reduce the spatial resolution and pass up the crucial information.
- 3) The image then goes through a series of residual connections which contains stacked convolution layers with shortcut connections.



- 4) The input in the residual blocks passes through Convolution then ReLU then Convolution.
- 5) Then the information from the original input and the information passed on from these layers are added and another ReLU is applied to the output. The identity or the shortcut connection helps the model learn the residuals meaning what has changed instead of learning the whole transformation from scratch.
- 6) After these residual blocks then go through layers like average pooling for flattening and then the fully connected layer is applied to make the final decision on the which class label it belongs to.

Vision Transformer B-16 Architecture



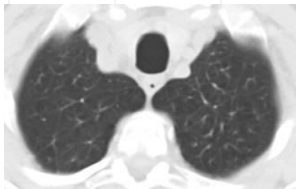
- 1) First we take an input image of size $224 \times 224 \times 3$.
- 2) Now here instead of applying convolutions, the image is divided into patches divided into fixed size patches lets say 16 resulting in $(224/16)^2 = 196$ patches.
- 3) Now each patch is flattened into a vector This is similar to like image patches are like words in a sentence.
- 4) Now each flattened patch is passed through a Linear Projection Layer to convert it to a fixed-dimensional embedding.

- 5) Now To preserve spatial information, a position embedding is added to each patch embedding.
- 6) These patch embedding are what is called tokens the above process is also called tokenization.
- 7) Now the Transformer encoder consists MLP which adds non-linearity to the model, Multi-head self attentions which allows the encoder to capture global relationships between patches, and Layer Normalization and skip connections stabilizes and speeds up training. This is repeated for L layers here it is 16 since B-16.
- 8) After the transformer layers, only the output corresponding to the token is taken this is the aggregated global feature of the image. This is passed to the MLP head for classification.

Datasets Used

SARs-CoV-2 Dataset

Class label	Number of images
Covid Positive	1252
Covid Negative	1230



Sample Covid
Negative



Sample Covid
Positive

Dataset link:

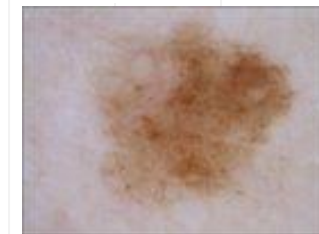
<https://www.kaggle.com/datasets/plameneduardo/sarscov2-ctscan-dataset>

ISIC 2016 Dataset

Class Label	Training No. of Images	Testing No. of Images
Benign	727	304
Melanoma	173	75
Total	900	379



Benign



Melanoma

Dataset link: <https://challenge.isic-archive.com/landing/2016/>

Performance Metrics Used

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$



Accuracy is the ratio of correctly predicted observations to the total observations. It measures the overall effectiveness of the model.

$$\text{Precision} = \frac{TP}{TP + FP}$$



Precision is the ratio of correctly predicted positive observations to the total predicted positives. It indicates the accuracy of the positive predictions.

$$\text{Specificity} = \frac{TN}{TN + FP}$$

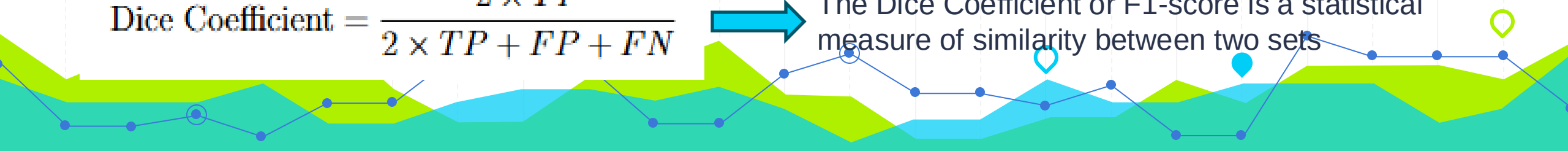


Specificity or Recall, also known as the true negative rate, measures the proportion of actual negatives that are correctly identified.

$$\text{Dice Coefficient} = \frac{2 \times TP}{2 \times TP + FP + FN}$$



The Dice Coefficient or F1-score is a statistical measure of similarity between two sets



Experimental Results

SARS-CoV-2 Dataset Basic Classifiers Result on 4 Random runs

Classifier	Mean Accuracy	Mean Precision	Mean Recall	Mean F1-score
Naive Bayes	71.85	84.05	56.53	67.58
KNN Classifier	93.94	92.18	95.97	94.02

ISIC 2016 Dataset Basic Classifier Result on 4 Random Runs

Classifier	Mean Accuracy	Mean Precision	Mean Recall	Mean F1-score
Naive Bayes	68.73	28.57	38.67	32.86
KNN Classifier	77.84	33.33	12.00	17.65

SARS-CoV-2 Dataset CNN models Result on 4 Random runs

Model	Mean Accuracy	Mean Precision	Mean Recall	Mean F1-score
ALexNET	92.08	91.40	93.08	92.21
Densenet-201	99.20	99.28	99.28	99.18
MobileNet-V2	91.78	90.73	93.25	91.98
ResNet-101	98.80	98.51	99.10	98.80
ViT-B-16	94.09	94.43	93.72	94.01

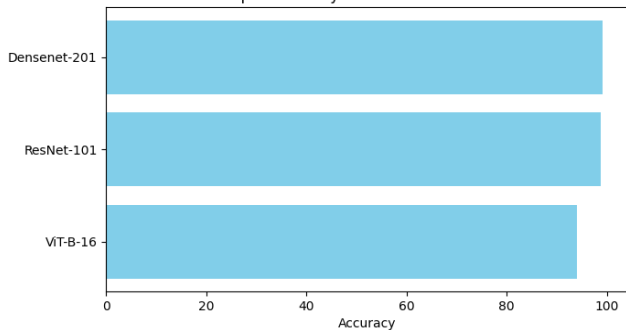
ISIC 2016 Dataset CNN Models Result on 4 Random Runs

Model	Mean Accuracy	Mean Precision	Mean Recall	Mean F1-score
ALexNET	82.79	88.02	90.96	89.44
Densenet-201	82.52	87.78	90.95	89.26
MobileNet-V2	80.15	87.14	88.32	87.69
ResNet-101	79.81	87.19	87.83	87.44
ViT-B-16	81.60	85.87	92.35	88.93

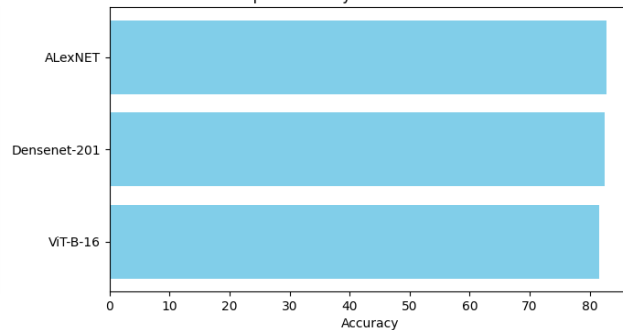


Best Outcomes

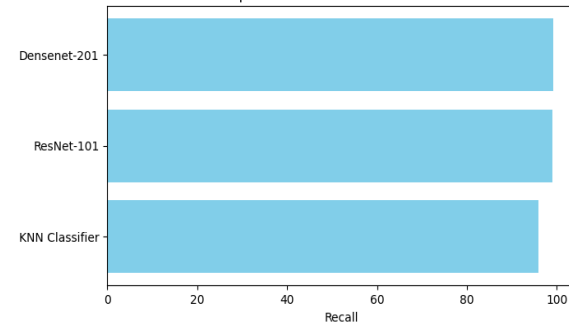
Top 3 Accuracy - SARS-CoV-2 Dataset



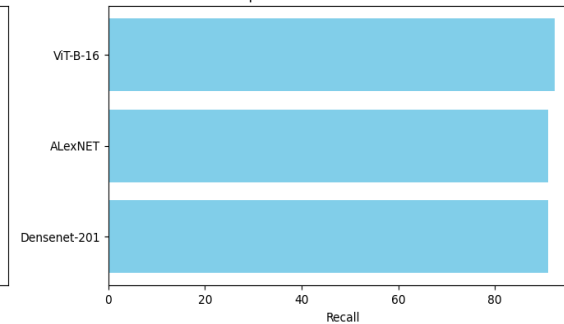
Top 3 Accuracy - ISIC 2016 Dataset



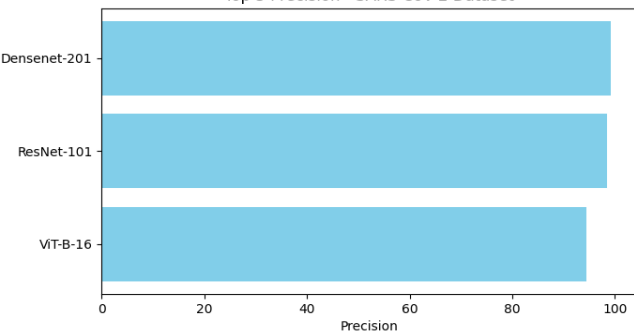
Top 3 Recall - SARS-CoV-2 Dataset



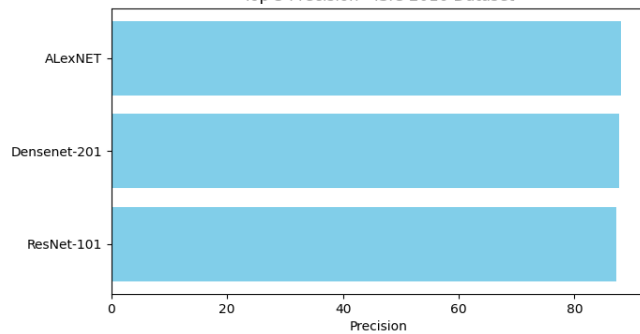
Top 3 Recall - ISIC 2016 Dataset



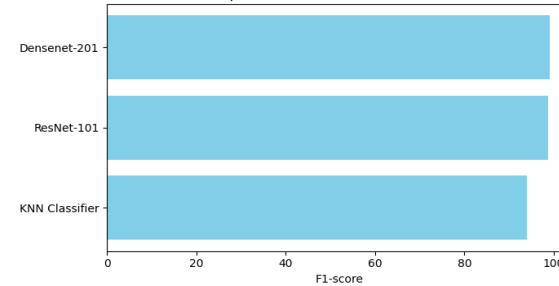
Top 3 Precision - SARS-CoV-2 Dataset



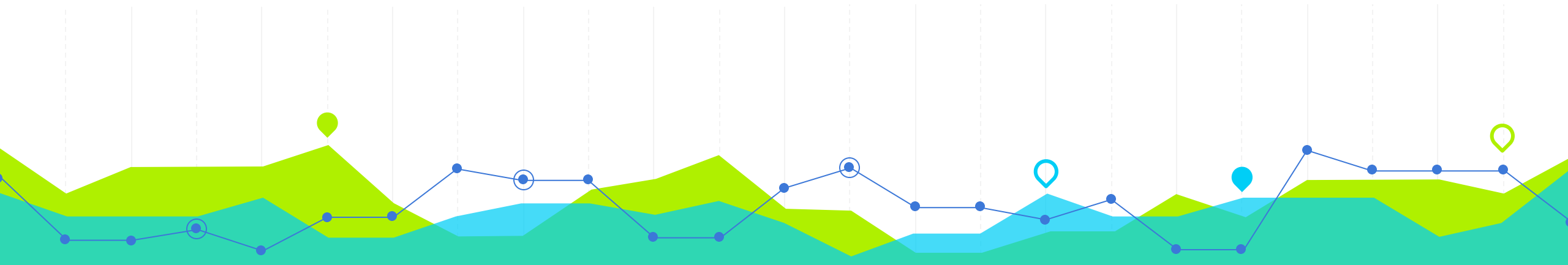
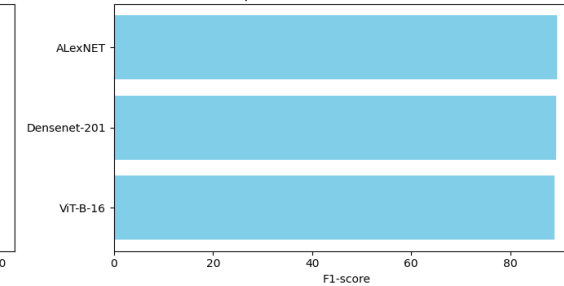
Top 3 Precision - ISIC 2016 Dataset



Top 3 F1-score - SARS-CoV-2 Dataset



Top 3 F1-score - ISIC 2016 Dataset



Conclusion

As seen from the bar graphs the CNN based models are dominant in every metrics top 3 performances and it is due to their detection of localized patterns using convolutions, max pooling techniques which preserve important details about the input data and then passing it on layer by layer which helps in prediction of the correct label even if the dataset has an imbalanced.

We see that the basic classifiers perform good in balanced dataset especially KNN due to calculation of Euclidean distances but once it becomes imbalanced the computation metrics such as precision, recall and F1-score get pretty low which show they are not able to a stable prediction on a highly imbalanced dataset such as the ISIC-2016.

Hence, through this project I can say that CNNs are better performer than the basic classifiers.



Further Opportunities and Ideas

These CNN models can be used in many ensemble based techniques like maximum voting technique which gives priority to a particular network let's say out of 3 networks one is given a weightage of 0.8 and the others are 0.1, 0.1.

Things like thresholding based segmentation can be performed on the images and then performing classification tasks using ensemble methods or individual CNN networks for better prediction of the class labels.



References

- 1) Class Notes from Slides on Classification Part 2 for Naïve Bayes and KNN classifier Algorithms.
- 2) ImageNet Classification with Deep Convolutional Neural Networks Alex Krizhevsky Ilya Sutskever Geoffrey E. Hinton.
- 3) MobileNetV2: Inverted Residuals and Linear Bottlenecks Mark Sandler Andrew Howard Menglong Zhu Andrey Zhmoginov Liang-Chieh Chen Google Inc.
- 4) Densely Connected Convolutional Networks Gao Huang* Cornell University, Zhuang Liu* Tsinghua University, Laurens van der Maaten Facebook AI Research, Kilian Q. Weinberger Cornell University
- 5) Deep Residual Learning for Image Recognition Kaiming He Xiangyu Zhang Shaoqing Ren Jian Sun Microsoft Research.
- 6) AN IMAGE IS WORTH 16X16 WORDS: TRANSFORMERS FOR IMAGE RECOGNITION AT SCALE Alexey Dosovitskiy*,† , Lucas Beyer* , Alexander Kolesnikov* , Dirk Weissenborn* , Xiaohua Zhai* , Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, Neil Houlsby*,† * equal technical contribution, † equal advising Google Research, Brain Team.
- 7) <https://www.kaggle.com/datasets/plameneduardo/sarscov2-ctscan-dataset>
- 8) <https://challenge.isic-archive.com/landing/2016/>



Thank YOU

